FlowDesk

List of Possible Objects

1. Project Identification

University	University of the Armed Forces ESPE
Department	Department of Computer Science
Course	Object-Oriented Programming (NRC: 36862)
Instructor	Edison Lascano, PhD
Project	FlowDesk — Business Management System
Software Company	Runtime Dynamics
Case Study	Imprenta Tumbaco
Team Members	Cristopher Lanchimba, Cristopher Guañuña, Julián Jaramillo

2. Purpose

This document identifies the possible objects of the FlowDesk application, detailing their main attributes, responsibilities, and relationships. FlowDesk is a local Java console prototype designed to manage the daily operations of a print shop using keyboard input and navigable menus. To keep the design organized, this catalog distinguishes the core **business objects** (which represent real-world concepts like customers and orders) from **utility classes** and **enumerations** (which support the application's infrastructure and logic).

3. Main Business Objects

The following table describes the core entities of the system. Note that text identifiers and descriptions use String, quantities use int, monetary values use double, and dynamically sized groups use ArrayList.

Class	Main Attributes	Responsibility
Customer	customerId, name, phone,	Represent customer

Class	Main Attributes	Responsibility
	idNumber, address	identification and contact information; support registration and searches.
User	userId, username, protectedPassword, role	Represent a user account, validate credentials, and identify the user's role. (Note: The prototype does not use password encryption or hashing).
Category	categoryId, name, description	Classify products and services.
CatalogItem	identifier, name, description, price, category	Intended abstract superclass for the integrated design. Groups the information and behavior shared by products and services.
Product	Inherited catalog attributes and product identification (productId)	Concrete specialization of CatalogItem in the intended integrated design. Represents a product offered by the print shop.
Service	Inherited catalog attributes and service identification (serviceId)	Concrete specialization of CatalogItem in the intended integrated design. Represents a service offered by the print shop.
WorkOrder	orderId, date, total, balance, status, customer, details, payments, user	Represent a customer job, contain its details, calculate totals and outstanding

Class	Main Attributes	Responsibility
		balance, and track its production status.
OrderDetail	detailId, itemName, quantity, description, unitPrice, associated product or service	Represent one requested item and calculate its subtotal. The integrated design may represent the selected item through a CatalogItem rather than separate product and service references.
Payment	paymentId, amount, date, paymentMethod, workOrder, cashMovement	Represent a partial or full payment and prevent invalid or repeated payment registration.
CashMovement	movementId, type, amount, date, reason, paymentMethod	Represent an income or expense and its effect on cash totals.
CashRegister	income, expenses	Accumulate income and expenses and calculate their difference.

4. Supporting Classes

These classes handle the application's interface, data processing, utilities, and errors.

Class	Responsibility
Flowdesk	Starts the application and coordinates authentication and role menus.
Dashboard	Displays order counts by status and pending orders.

Class	Responsibility
Report	Prepares operational and financial report information. The integrated design includes period selection and CSV export.
DataManager	Loads and saves JSON data and resolves the data directory. Complete JSON recovery is intended for the integrated design.
ConsoleInput	Reads keyboard input, retries invalid entries, and supports cancellation.
Validation	Centralizes required-field, numeric, and identifier checks.
Date	Represents day, month, and year (the standard LocalDate is not used in this prototype).
ValidationException	Reports business validation errors.
InvalidPaymentException	Specializes ValidationException for payment errors.
PersistenceException	Reports failures when reading or saving data.

Enumerations

The system uses the following enumerations. The internal identifiers are in English, but the application displays Spanish labels to the customer (e.g., WAITING is shown as "En espera", CASH as "Efectivo").

- **Role:** MANAGER, CASHIER, EMPLOYEE.
- **OrderStatus:** WAITING, APPROVED, IN_PROGRESS, COMPLETED, DELIVERED.
- **PaymentMethod:** CASH, TRANSFER.
- **MovementType:** INCOME, EXPENSE.
- **ReportType:** DAILY, WEEKLY, MONTHLY.

5. Main Relationships

Objects in FlowDesk collaborate constantly. The table below summarizes how the main classes

interact:

Relationship	Description
Customer ↔ WorkOrder	One customer can have several work orders. Each registered business order belongs to one customer.
User ↔ WorkOrder	One user can create several orders. (Note: Manager, Cashier, and Employee are roles of User, not separate subclasses).
WorkOrder ↔ OrderDetail	A registered order contains one or more details.
WorkOrder ↔ Payment	One order can have zero or more payments. Each payment belongs to one order.
Payment ↔ CashMovement	A payment processed through the cash workflow generates one related income movement. Manual cash movements do not require an order payment.
Category ↔ CatalogItem	A category classifies several products and services.
Dashboard / Report → Data	Dashboard and Report use operational records to generate their views.
DataManager → Entities	DataManager reads and writes the stored information for all business entities.

Generalization Hierarchy

In the intended integrated design, Product and Service share a common parent class. This demonstrates the difference between an **"is-a"** relationship and a **"has-a"** relationship: A Product *is a* CatalogItem, whereas a WorkOrder *has* OrderDetails.

```
CatalogItem
├── Product
└── Service
```

Important Business Rules

- Required fields cannot be empty.
- Identifiers must be unique within their record type.
- Quantities must be positive integers.
- Prices must be finite and non-negative.
- Payments must be positive and cannot exceed the outstanding balance.
- A payment cannot be registered twice.
- Orders cannot be registered without details.

Core Formulas

- **Detail subtotal** = quantity × unit price.
- **Order total** = sum of detail subtotals.
- **Outstanding balance** = order total − recorded payments.
- **Cash balance** = income − expenses.

Collections Clarification

The prototype uses structures like ArrayList<Customer>, ArrayList<WorkOrder>, and ArrayList<Payment> to store dynamically sized groups of business objects. When primitive values need to be stored in collections, they are wrapped (e.g., ArrayList<Integer> and ArrayList<Double>). It is important to note that an ArrayList contains wrapper objects, not primitive arrays, and therefore does not automatically satisfy a requirement specifically asking for basic primitive array implementation.

6. Conclusion

This catalog defines the essential structural foundation for the FlowDesk application. By separating core business logic (like orders and payments) from utilities (like JSON persistence and console input), the design adheres to Object-Oriented Programming principles. The intended integration of generalization through CatalogItem will further reduce code duplication, making the Unit 1 prototype robust and ready for future expansions.